SMART CITY ANALYTICS COMPANY

REPORT

Prepared by

Hala AlJafary
Hiba Daher
Shahed Abu Baidar

Part-1: The Multi-Source ETL Process

Task 1.1: Extract - Transit Data & Sensor Data Ingestion

A. Public Transit Data Extraction:

What happens:

- Sends HTTP requests
- Parses responses
- Converts raw data into structured format

Steps:

1. Request data from online source
2. Parse HTML / JSON
3. Store results in DataFrames

Challenges:

- Missing or malformed data or unavailable columns:

During the data extraction phase from Numbeo, not all required transit-related attributes were directly available in the source tables.
Specifically, columns such as:

- Transit_Score
- Ridership_Millions
- Annual_Budget_USD
- System_Age_Years
- Avg_Delay_Minutes

Transit line details (lines, stations, passengers)

were not provided explicitly by the data source.

Approach Taken:

Instead of discarding the dataset or relying on incomplete information, new columns were engineered using logically related existing features.

The process followed these steps:

1. Identification of Relevant Existing Columns

   Columns such as:

   - Traffic Commute Time Index
   - Pollution Index
   - Safety Index
   - Quality of Life Index
   - Cost of Living Index
   - Property Price to Income Ratio

2. Normalization of Features

   - MinMaxScaler was applied to bring all selected features to a common scale.
   - Inverse normalization was used where lower values represent better performance (e.g., commute time and pollution).

3. Weighted Feature Engineering

   - New metrics were calculated using weighted combinations of normalized features.
   - Weights were chosen to reflect realistic influence (e.g., commute time has higher impact on transit efficiency).

Examples of Derived Columns:

- Transit_Score
  Derived from:
  - Traffic Commute Time Index (50%)
  - Pollution Index (30%)
  - Safety Index (20%)

- Ridership_Millions
  Estimated using:

  o Commute Time

  - Quality of Life
  - Cost of Living

- Annual_Budget_USD
  Approximated based on:

  o Estimated ridership

  - Cost of living

- System_Age_Years
  Estimated using:

  - Property price pressure

  o Commute time intensity

- Transit Lines & Stations
  Since line-level data was unavailable:

  - Synthetic data was generated using realistic ranges.

  o This allows relational database modeling and downstream analytics.

```
[47]  print(transit_systems.head())

...      City_ID             City_Name      Country  Transit_Score  \
     0        1  The Hague (Den Haag)  Netherlands      93.079177
     1        2             Groningen  Netherlands      92.495935
     2        3             Eindhoven  Netherlands      90.196619
     3        4                  Bern  Switzerland      78.267242
     4        5               Utrecht  Netherlands      89.008585

        Avg_Delay_Minutes  Ridership_Millions  Annual_Budget_USD  System_Age_Years
     0               8.24            0.431991       1.108991e+08             15.52
     1               8.04            0.405214       1.015214e+08             14.42
     2               9.08            0.437971       1.100971e+08             17.54
     3              13.80            0.620134       1.654134e+08             24.70
     4               9.04            0.429186       1.119186e+08             21.32
     <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 279 entries, 0 to 278
```

```
[48]  print(transit_lines.head())

...    City_ID                  Line_Name      Line_Type  Stations_Count  \
    0        1  The Hague (Den Haag) Line 1  Commuter Rail              34
    1        1  The Hague (Den Haag) Line 2           Tram              44
    2        2              Groningen Line 1          Metro              46
    3        2              Groningen Line 2           Tram              21
    4        2              Groningen Line 3           Tram              23

       Daily_Passengers
    0            472894
    1             83523
    2            880045
    3            608135
    4             38251
```

B. Infrastructure Sensor Data Extraction:

The sensor data was expected to be retrieved, However, during execution, the API did not return valid data due to connectivity issues, timeout, or service unavailability.
Steps Taken**:**

1. Attempted Live API Extraction

   o A GET request is sent to the sensor API endpoint with a timeout constraint.

   o The response status is validated using raise_for_status().

   o If successful, the returned JSON is converted into a pandas DataFrame.

   o Missing values are replaced with zeros to ensure data consistency.

2. Exception Handling & Detection of Failure

   o Any network-related error (timeout, unreachable server, invalid response) is caught using RequestException.

   o This indicates that no real sensor data was available at runtime.

3. Fallback Data Generation

   Since the API failed to return data, a synthetic dataset was generated to ensure pipeline continuity:

   o sensor_id: Sequential identifiers

   o timestamp: Simulated real-time data using minute offsets

   o temperature: Random values within realistic operational ranges

- humidity: Random values representing environmental conditions
- status: Probabilistic system status (OK / FAIL)
- vibration: Random sensor vibration values

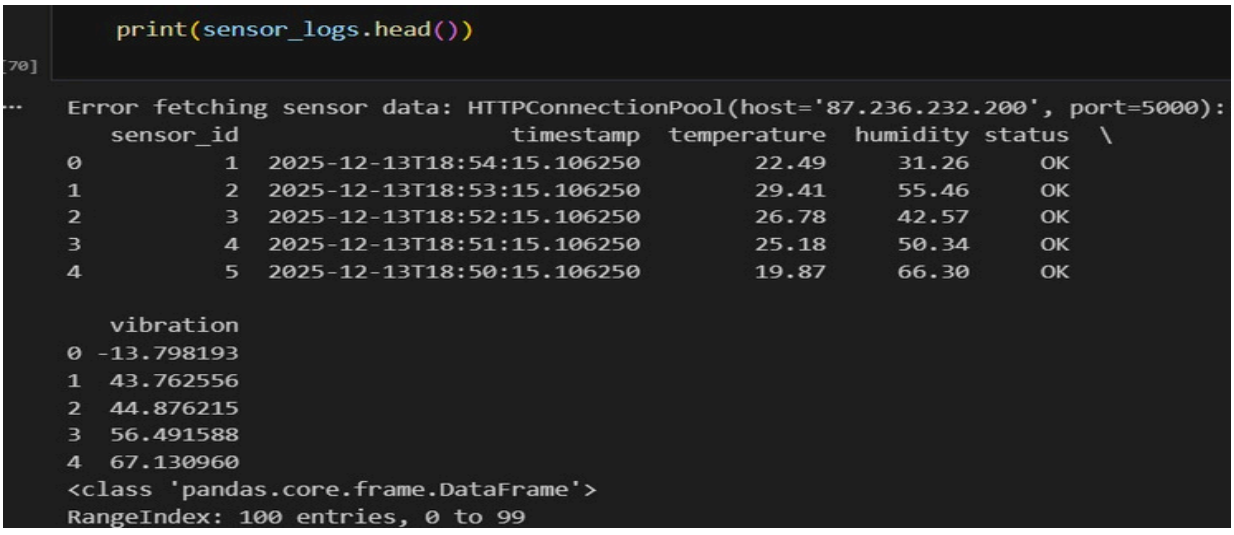
A fixed random seed (np.random.seed(42)) was used to ensure reproducibility.

Challenges Encountered:

- External API dependency introduced instability
- Live data availability could not be guaranteed

Insights Gained:

- Implementing fallback logic is critical for resilient ETL pipelines
- Synthetic data enables:
  - End-to-end pipeline testing
  - Database schema validation
  - Machine learning workflow continuity



Steps Taken:

- Handle null values
- Normalize formats
- Create new features
- Convert data types

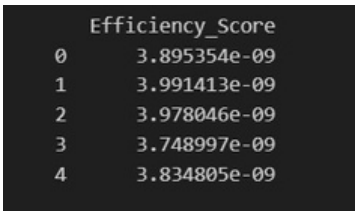
Challenges:

- Inconsistent units
- Unexpected missing values

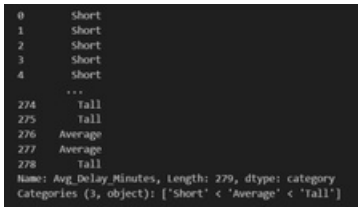
Insights:

Data quality directly impacts ML performance and analytics accuracy.

Some examples of steps taken:



	Efficiency_Score
0	3.895354e-09
1	3.991413e-09
2	3.978046e-09
3	3.748997e-09
4	3.834805e-09



```
0        Short
1        Short
2        Short
3        Short
4        Short
        ...
274       Tall
275       Tall
276    Average
277    Average
278       Tall
Name: Avg_Delay_Minutes, Length: 279, dtype: category
Categories (3, object): ['Short' < 'Average' < 'Tall']
```

Task 1.3: Load - Urban DataWarehouse

Purpose: Establishes a connection between Python andthe SQL database.

Steps Taken:

- Create SQLAlchemy engine
- Enable communication between pandas and SQL

Challenges:

- Incorrect credentials or database URI will break the pipeline
- SQL injection risks if raw queries are not handled carefully

Insight:

Using SQLAlchemy provides flexibility and database independence.

```
[1236]: pd.read_sql("SELECT * FROM transit_systems LIMIT 5", engine)
```

[1236]:

	City_ID	City_Name	Country	Transit_Score	Avg_Delay_Minutes	Avg_Delay_Minute_category	Ridership_Millions	Annual_Budget_USD	Syst
0	1	The Hague (Den Haag)	netherlands	93.0792	8.24	Short	0.431991	110899000.0	
1	2	Groningen	netherlands	92.4959	8.04	Short	0.405214	101521000.0	
2	3	Eindhoven	netherlands	90.1966	9.08	Short	0.437971	110097000.0	
3	4	Bern	switzerland	78.2672	13.80	Short	0.620134	165413000.0	
4	5	Utrecht	netherlands	89.0086	9.04	Short	0.429186	111919000.0	

Activate Windows
Go to Settings to activate Windows.

```
[11]: pd.read_sql("SELECT * FROM transit_lines LIMIT 5", engine)
```

[11]:

	Line_ID	City_ID	Line_Name	Line_Type	Stations_Count	Daily_Passengers
0	1969	1	The Hague (Den Haag) Line 1	Bus	21	644736
1	1970	1	The Hague (Den Haag) Line 2	Bus	24	188031
2	1971	2	Groningen Line 1	Bus	17	76234
3	1972	2	Groningen Line 2	Tram	40	419995
4	1973	3	Eindhoven Line 1	Metro	23	964443

```
[14]: pd.read_sql("SELECT * FROM sensor_logs LIMIT 5", engine)
```

[14]:

	Sensor_ID	Timestamp	Temperature	Humidity	Status	Vibration	hour	day_of_week
0	2	2025-12-25 23:43:03	0.979807	55.46	OK	0.438213	23	Thursday
1	3	2025-12-25 23:42:03	0.748903	42.57	OK	0.449568	23	Thursday
2	4	2025-12-25 23:41:03	0.608428	50.34	OK	0.568000	23	Thursday
3	5	2025-12-25 23:40:03	0.142230	66.30	OK	0.676480	23	Thursday
4	6	2025-12-25 23:39:03	0.142230	39.97	OK	0.982071	23	Thursday

Part-2: Analytics & Predictive Modeling

Task 2.1: SQL Analytics on Urban Data

Purpose

This function is responsible for generating actionable insights from the processed and stored data in the UrbanIQ_Warehouse database.
It applies analytical SQL queries to evaluate transit efficiency, service delays, and system maturity, then exports the results into a daily report.

Query 1: Top 5 Transit Efficiency Cities

What This Query Does

- Computes a Transit Efficiency Score by dividing ridership by annual budget.
- Ranks cities by how effectively financial resources are converted into ridership.
- Retrieves the top 5 most efficient cities.

Query 2: Average Delay by Transit Line Type

What This Query Does

- Joins line-level data with city-level system data.
- Calculates the average delay per transit mode (Metro, Bus, Tram, etc.).

Query 3: High-Performing but Aging Transit Systems

What This Query Does

- Identifies cities with:
    - High transit performance
    - Aging infrastructure
- Uses dynamic thresholds instead of hardcoded values.

Query1

```
[1251]: query1 = """
        SELECT City_Name, Ridership_Millions / Annual_Budget_USD AS efficiency_Score
        FROM transit_systems
        ORDER BY efficiency_Score DESC
        LIMIT 5;"""

        df_top5_efficiency = pd.read_sql(query1, engine)
        print(df_top5_efficiency, "\n")
```

```
    City_Name  efficiency_Score
0     Kolkata      7.331637e-09
1  Coimbatore      7.227319e-09
2   Bangalore      7.089402e-09
3       Delhi      7.042894e-09
4      Jaipur      6.924552e-09
```

Query2

```
[1257]: query2 = """
        SELECT transit_lines.Line_Type,
        AVG (transit_systems.Avg_Delay_Minutes) AS Avg_Delay
        FROM transit_lines
        JOIN transit_systems ON transit_lines.City_ID = transit_systems.City_ID
        GROUP BY transit_lines.Line_Type;"""

        df_delay_by_type = pd.read_sql(query2, engine)
        print(df_delay_by_type)
```

```
       Line_Type  Avg_Delay
0            Bus  14.160637
1           Tram  14.144127
2          Metro  14.494387
3  Commuter Rail  14.673509
```

Activate Windows

Query3

```
[1260]: high_score = transit_systems['Transit_Score'].quantile(0.75)
        old_age = transit_systems['System_Age_Years'].quantile(0.75)

        query3 = f"""
        SELECT City_Name, Transit_Score, System_Age_Years
        FROM transit_systems
        WHERE Transit_Score > {high_score}
          AND System_Age_Years > {old_age};"""

        df_high_old = pd.read_sql(query3, engine)
        print(df_high_old)
```

```
         City_Name  Transit_Score  System_Age_Years
0           Prague        75.2052             41.98
1           Malaga        79.2865             39.12
2            Split        89.0584             42.18
3       Bratislava        73.9048             37.84
4  Nizhny Novgorod        78.4884             36.82
```

Activate Windows
Go to Settings to activate Windows.

Machine Learning Phase: Introduces the ML section of the pipeline.

Train/Test Split: Split dataset into training and testing sets.

Handling Class Imbalance (SMOTE): Imbalanced data leads to biased predictions.

Model Selection: Explains why Logistic Regression was chosen.

Model Evaluation Strategy: Discusses evaluation metrics and validation.

Model Training: Initialize model and train on balanced data.

```
print("Starting Machine Learning Pipeline: Training Health Model...")
ddf = sensor_logs[['sensor_id','temperature','humidity','vibration','hour','status']].copy()
ddf['status'] = ddf['status'].map({'OK': 1, 'FAIL': 0})

y = ddf['status']
x = ddf.drop(columns=['status'])

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

smote = SMOTE(random_state=42, k_neighbors=3)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

model = LogisticRegression()
model.fit(X_train_smote, y_train_smote)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print(f"Model Retrained. Accuracy: {accuracy}, F1 Score: {f1} ,Precision :{precision},Recall:{recall}")

dump(model, 'model.joblib')
print("Model saved to model.joblib")
loaded_model = load('model.joblib')
y_pred_new = loaded_model.predict(X_test)
print("Model verified successfully. Ready for deployment.")
```

Insight:

- Logistic Regression is interpretable and efficient.
- SMOTE improves recall for minority classes.
- Ensures fair evaluation of model performance.

Part 3: Pipeline Orchestration with Apache Airflow

Purpose

This DAG orchestrates the entire UrbanIQ monitoring and analytics pipeline, ensuring automated, reliable, and near-real-time execution of data extraction, transformation, loading, analytics, and machine learning tasks.

DAG Configuration (Default Arguments)

The default_args dictionary defines global execution behavior for all tasks:

- owner: Identifies the pipeline maintainers (hala,shahed, hiba)
- depends_on_past: Disabled to prevent task blocking due to previous failures such as email_on_failure ,on_failure_callbac.
- start_date: Specifies when the DAG becomes active (19,12,2025).
- email & notifications: Enables alerting on task failure and send it to ['hlam41102@gmail.com' ,'shahadabubaydar@gmail.com' ,'hibaadaher005@gmail.com'].
- retries & retry_delay: Improves fault tolerance by retrying failed tasks automatically
  The system is configured to perform 3 automatic retries in the event of an error.
  A 5-minute retry delay has been set between attempts to enhance reliability.

Insight:
These configurations ensure resilience, traceability, and operational reliability)on_failure_callback(to send immediate automated alerts whenever a task fails, ensuring a rapid response to any system malfunctions.
Scheduling Strategy

- Executes the pipeline every 6 hours.
- Supports near-real-time urban monitoring without overloading resources.
- catchup=False prevents retroactive execution for past dates.
- Monthly ML Logic: the `train_health_model` task is programmed to execute its full training logic only once a month.

```python
default_args = {
    'owner': 'Hala_Hiba_Shahed',
    'depends_on_past': False,
    'start_date': datetime(2025, 12, 19),
    'email': ['hlam41102@gmail.com','shahadabubaydar@gmail.com','hibaadaher005@gmail.com'],
    'email_on_failure': True,
    'retries': 3,
    'retry_delay': timedelta(minutes=5),}

with DAG('urbaniq_monitoring_pipeline',
    default_args=default_args,
    description='Near-real-time urban monitoring and ML pipeline',
    schedule_interval='0 */6 * * *',
    catchup=False) as dag:
```

Activate Windows

Task Breakdown

Data Extraction Tasks (Parallel Execution)

- extract_transit_data
- extract_sensor_data

Purpose:
Retrieve data from heterogeneous sources (web, APIs)

)Fallback Mechanism (The system is programmed to generate synthetic data if the sensor server is unavailable, ensuring that the pipeline remains fully operational.

```python
def extract_transit_data():
 try:
      print("Fetching data from Numbeo...")
```

```python
 except Exception as e:
     print(f"!!! ALERT: Failure in Extraction Task !!! Sending notification to team...")
     print(f"Error Details: {str(e)}")
     raise e
```

```python
def extract_sensor_data(url="http://87.236.232.200:5000/data", fallback_rows=100):
    try:
        np.random.seed(42)
```

```python
    except requests.exceptions.RequestException as e:
        print(f"!!! ALERT: Sensor API is unreachable !!!.")
        print(f"Details: {e}")
```

Data Transformation

- transform_urban_data

Purpose:
Cleans, normalizes, and integrates extracted datasets into a unified structure.
)Data Quality Check(The system now validates data thresholds, such as missing value percentages and extreme temperature readings, issuing alerts if any anomalies or corrupted data are detected.

```
def transform_urban_data(transit_systems, sensor_logs):
 try:
  print("Starting Data Transformation...")
```

```
  NULL_THRESHOLD = 0.20
  TEMP_MIN, TEMP_MAX = -10, 50
  VIB_MAX = 100
  null_ratio = sensor_logs[['temperature', 'humidity', 'vibration']].isnull().mean().max()
  extreme_temp = sensor_logs[(sensor_logs['temperature'] < TEMP_MIN) | (sensor_logs['temperature'] > TEMP_MAX)].any().any()
  extreme_vib = (sensor_logs['vibration'] > VIB_MAX).any()

  if null_ratio > NULL_THRESHOLD or extreme_temp or extreme_vib:
        print(f"!!! ALERT: Data Quality Drop Detected !!!")
        if null_ratio > NULL_THRESHOLD:
            print(f"- High number of missing values: {null_ratio:.2%}")
        if extreme_temp:
            print(f"- Out-of-range temperature detected!")
        if extreme_vib:
            print(f"- Abnormal vibration levels detected!")
        print(f"Simulating Alert Email sent to the team: Hala_Hiba_Shahed")
  else:
        print("Data Quality Check Passed: All sensor readings are within normal thresholds.")
  print("Data transformed successfully.")
  return transit_systems,sensor_logs
 except Exception as e:
        print(f"!!! CRITICAL ALERT: Transformation Task Failed !!!")
        print(f"Error Details: {e}")
        raise e
```

Data Loading

- load_data_to_warehouse

Purpose:
Stores processed data into the UrbanIQ_Warehouse for analytical querying
)Idempotency(The code ensures data consistency by preventing duplication during re-runs. It utilizes "Truncate and Load" logic (clearing existing tables before inserting new data), which maintains overall database integrity.

```
    with engine.connect() as conn:
            conn.execute(text(stmt_transit_systems))
            conn.execute(text(stmt_transit_lines))
            conn.execute(text(stmt_sensor_logs))
            conn.execute(text("SET FOREIGN_KEY_CHECKS = 0;"))
            conn.execute(text("DELETE FROM transit_lines"))
            conn.execute(text("DELETE FROM transit_systems"))
            conn.execute(text("DELETE FROM sensor_logs"))
            conn.execute(text("SET FOREIGN_KEY_CHECKS = 1;"))
            conn.commit()

    transit_systems.to_sql( name='transit_systems',con=engine, if_exists='append', index=False)
    transit_lines.to_sql(name='transit_lines',con=engine,if_exists='append',index=False)
    sensor_logs.to_sql(name='sensor_logs',con=engine,if_exists='append',index=False)

    print("Data loaded successfully. Idempotency ensured.")

except Exception as e:
    print(f"!!! CRITICAL ALERT: Load Task Failed !!!")
    print(f"Error Details: {e}")
    raise e
```

Downstream Parallel Tasks

- generate_city_insights
- train_health_model

Purpose:

- Generate analytical reports
- Train ML models using the latest data snapshot:

)Smart Alerts(Logic has been implemented to send alerts if the model's **accuracy** drops below **40%**.

```
    with open(report_filename, 'w') as f:
        f.write(f"--- Smart City Daily Report: {today_str} ---\n\n")
        f.write("1. Top 5 Transit Efficiency:\n")
        f.write(df_top5_efficiency.to_string() + "\n\n")
        f.write("2. Average Delay by Line Type:\n")
        f.write(df_delay_by_type.to_string() + "\n\n")
        f.write("3. High Performing & Established Systems:\n")
        f.write(df_high_old.to_string() + "\n")

    print(f"Report generated successfully: {report_filename}")
except Exception as e:
    print(f"!!! ALERT: Failed to generate City Insights !!!")
    print(f"Error Details: {e}")
    raise e
```

```
    print(f"Model Retrained. Accuracy: {accuracy}, F1 Score: {f1} ,Precision :{precision},Recall:{recall}")
    if accuracy < 0.40:
        print(f"!!! ALERT: Model Performance is below threshold (Acc: {accuracy:.2f}) !!!")
        print("Simulating Email to Data Science Team: Model needs investigation.")
```

Execution Flow & Task Dependencies

To ensure data integrity and prevent process overlap, the task sequence is designed as follows:

1. Parallel Extraction: The tasks extract_transit_data and extract_sensor_data are triggered simultaneously to optimize performance and reduce total execution time.
2. Sequential Processing: The transform_urban_data task acts as a checkpoint; it only begins after both extraction tasks succeed. This is followed by load_data_to_warehouse to ensure no raw or uncleaned data enters the database.
3. Final Output: In the final stage, the pipeline splits again to execute generate_city_insights and train_health_model in parallel, utilizing the finalized and verified data.

Why this sequence ?

- Data Integrity: Prevents the analysis of incomplete or missing data.
- Efficiency: Minimizes total runtime by running independent tasks in parallel.
- Reliability: The pipeline automatically halts at any point of failure before affecting the data warehouse, ensuring a "Fail-Safe" environment.

Challenges Encountered:

- External Data Source Reliability
- Workflow Dependency Management
- Scheduling Trade-offs
- Error Propagation Risk
- Scalability Considerations